

Оптимизация SQL запроса – это процесс анализа, который позволяет определить серверу какие ресурсы будут использованы при выполнении запроса, каким образом можно уменьшить его ресурсоемкость и сократить время его выполнения.

Для выбора наилучшего способа выполнения запроса, оптимизатор использует информацию оптимизации о том, к каким объектам базы данных обращается запрос, каковы их размеры и путь доступа к ним, наличие и тип индексов таблиц, плотность и распределения данных в таблицах, политику использования кэша и количество запрашиваемых данных.

Оптимизатор Adaptive Server

Оптимизатор – это подсистема ASE, который на основе информации об объектах базы данных, создает план анализируемого запроса, впоследствии используемый для доступа к данным.

Оптимизатор изучает разобранные (синтаксически) и нормализованные запросы, а также информацию об объектах базы данных. Входные данные для оптимизатора – это разобранный (синтаксически) запрос SQL и статистика по таблицам, индексам и столбцам, упомянутым в запросе.

Выходными данными оптимизатора является план запроса.

План запроса – это скомпилированный код, содержащий упорядоченные этапы выполнения запроса, включая методы доступа (сканирование таблицы или сканирование индекса, тип соединения, порядок соединения и т.д.) для каждой таблицы.

Используя статистику по таблицам и индексам, оптимизатор прогнозирует стоимость применения альтернативных методов доступа для разрешения определенного запроса. Он находит лучший план запроса – план с наименьшей стоимостью ввода-вывода. Для многих запросов существует несколько возможных планов. Adaptive Server выбирает наиболее дешевый план, компилирует и выполняет его.

Этапы обработки запроса

Обработка запроса сервером Adaptive Server делится на следующие этапы:

1. Запрос разбирается и нормализуется. Синтаксический анализатор удостоверяет правильность синтаксиса SQL. Нормализация удостоверяет, что все объекты, на которые ссылается запрос, существуют. Делается проверка разрешений, чтобы убедиться, что у пользователя есть полномочие на обращение ко всем таблицам и столбцам запроса.
2. Предварительная обработка приводит некоторые аргументы поиска в оптимизированную форму и добавляет оптимизированные аргументы поиска и инструкции соединения.
3. По мере оптимизации запроса каждая его часть анализируется, и выбирается лучший план

запроса. Оптимизация включает следующие действия:

- анализ каждой таблицы;
- оценку стоимости использования каждого индекса, который соответствует аргументу поиска или столбцу, по которому производится соединение;
- выбор порядка и типа соединения;
- определение окончательного метода доступа.

4. Выбранный план запроса компилируется.

5. Запрос выполняется, и результаты возвращаются пользователю.

Работа с оптимизатором

Цель оптимизатора – выбрать для каждой таблицы метод доступа, который сократит общее время обработки запроса. Выбор оптимизатора базируется на имеющейся статистике по таблицам, к которым обращается запрос, и других факторах, таких как стратегии кэширования, размер кэша и размер блока ввода-вывода. Главным компонентом, на котором основывается принятие решений оптимизатором, является доступная статистика по таблицам, индексам и столбцам.

В некоторых ситуациях может показаться, что оптимизатор делает неправильный выбор методов доступа. Это может быть результатом предоставления неточной или неполной информации (например, устаревшей статистики). В других случаях дополнительный анализ и использование специальных возможностей обработки запросов может определить источник проблемы и найти ее решения или обходные пути.

В качестве меры стоимости запроса оптимизатор использует стоимость ввода-вывода.

Основные компоненты стоимости обработки запроса:

- физический ввод-вывод, когда страницы должны быть считаны с диска;
- логический ввод-вывод, когда страницы для запроса должны быть считаны из кэша.

Оптимизация запросов

Чтобы понять оптимизацию запроса и определить, возможно ли повысить его производительность, необходимо изучить, как запрос обращается к объектам базы данных, а также знать, какие размеры имеют объекты и какие индексы построены по таблицам.

Ниже приведены некоторые симптомы проблем с оптимизацией:

- Запрос выполняется медленнее, чем должен при данных индексах и размере таблицы.
- Запрос выполняется медленнее, чем аналогичные запросы.
- Запрос неожиданно начинает выполняться медленнее, чем обычно.

- Обработка запроса в хранимой процедуре занимает больше времени, чем выполнение его как нерегламентированного оператора.
- План запроса показывает, что выполняется сканирование таблицы, хотя вы ожидали использования индекса.

Вот некоторые источники проблем с оптимизацией:

- Статистика давно не обновлялась, поэтому реальное распределение данных не соответствует значениям, которые Adaptive Server использует для оптимизации запросов.
- Строки, к которым должна обратиться данная транзакция, не соответствуют статистике индексов.
- Для доступа к большому фрагменту таблицы используется индекс.
- Инструкции `where` записаны в форме, которая не может быть оптимизирована.
- Для важного запроса не существует подходящего индекса.
- Хранимая процедура была скомпилирована до того, как в основные таблицы были внесены значительные изменения.

Факторы, которые изучаются во время оптимизации

Планы запросов состоят из тактики извлечения необходимых запросу данных и упорядоченного набора этапов выполнения. Разрабатывая планы запроса, оптимизатор изучает следующие моменты:

- Размер каждой таблицы в запросе (как количество строк, так и количество страниц данных) и количество страниц индексов и распределения памяти, которые должны быть считаны.
- Существующие индексы по таблицам и столбцам, которые используются в запросе, тип и количество уровней индекса, количество листовых страниц и коэффициенты кластеризации для каждого индекса.
- Покрывается ли запрос индексом, то есть может ли запрос быть выполнен путем извлечения данных из листовых страниц индекса без доступа к страницам данных. Adaptive Server может использовать индексы, которые покрывают запросы, даже если инструкции **where** в запрос не включены.
- Плотность и распределение ключей в индексах.
- Размер доступного кэша или кэш-данных, размер блока ввода-вывода, поддерживаемый кэшами, и планируемая стратегия кэширования.
- Стоимость операций физического и логического чтения.
- Инструкции соединения и самый лучший порядок и тип соединения. При этом принимается во внимание стоимость и количество операций сканирования, необходимых для каждого соединения, и полезность индексов при ограничении ввода-вывода.
- Будет ли построение рабочей таблицы (внутренней временной таблицы) с индексом по столбцам соединения выполнено быстрее, чем неоднократные сканирования таблицы, если для внутренней таблицы соединения нет полезных индексов.
- Содержит ли запрос агрегатную функцию `max` или `min`, которая может применять индекс

для нахождения значения без сканирования таблицы.

- Будут ли страницы данных или индекса повторно требоваться для выполнения такого запроса, как соединение, или можно использовать стратегию “выборка и удаление”, так как страницы потребуются только один раз.

Для каждого плана оптимизатор определяет полную стоимость, подсчитывая стоимость логического и физического ввода-вывода. Затем Adaptive Server использует план с наименьшей стоимостью.

Хранимые процедуры и триггеры оптимизируются, когда объект выполняется в первый раз, при этом план запроса сохраняется в кэше процедур. Если другие пользователи выполняют ту же процедуру, когда неиспользованная копия плана находится в кэше, в кэш копируется скомпилированный план запроса, а план, находящийся в кэше, не перекомпилируется.

Предварительная обработка может добавить инструкции для оптимизации

После того как запрос уже разобран и нормализован, но до того как оптимизатор начнет его анализ, этот запрос обрабатывается, чтобы увеличить количество инструкций, которые могут быть оптимизированы. Эта обработка состоит в следующем:

- некоторые аргументы поиска преобразуются в эквивалентные аргументы;
- некоторые выражения, используемые как аргументы поиска, предварительно вычисляются, и находится литеральное значение, которое можно оптимизировать;
- по возможности применяется транзитивное замыкание аргументов поиска;
- по возможности применяется транзитивное замыкание столбцов, по которым производится соединение;
- для некоторых запросов, в которых используется ключевое слово `or`, могут быть сформированы дополнительные аргументы поиска, что создает дополнительные варианты для оптимизации.

Преобразование инструкций в эквивалентные аргументы поиска

На этапе предварительной обработки производится поиск инструкций запроса, которые могут быть преобразованы в форму аргументов поиска (search arguments, SARG).

Эквивалентные аргументы поиска

Инструкция	Преобразование
between	Преобразуется в инструкции >= и <= . Например, условие between 10 and 20 преобразуется в >= 10 and <= 20
like	Если первый символ шаблона – константа, инструкции like могут быть преобразованы в условия сравнения “больше” или “меньше”. Например, условие like 'sm%' эквивалентно >= 'sm' and < 'sn' . Если первый символ – это метасимвол, то такая инструкция, как like '%x' , не может использовать индекс для обработки, но для подсчета количества соответствующих шаблону строк могут использоваться значения гистограммы.
in (список_значений)	Преобразуется в список запросов or . Например, условие int_col in (1, 2, 3) становится int_col = 1 or int_col = 2 or int_col = 3

Преобразование выражений в аргументы поиска

Прежде чем начнется оптимизация запроса, многие выражения будут преобразованы в литеральные строки поиска. В следующих примерах приведены обработанные выражения (в том виде, в каком они показаны в блоке анализа аргументов поиска):

Операция	Пример инструкции where	Обработанное выражение
Неявное преобразование	<code>numeric_col = 5</code>	<code>numeric_col = 5.0</code>
Функция преобразования	<code>int_column = convert(int, '77')</code>	<code>int_column = 77</code>
Арифметические действия	<code>salary = 5000*12</code>	<code>salary = 600000</code>
Математические функции	<code>width = sqrt(900)</code>	<code>width = 30</code>
Строковые	<code>shoe_width</code>	<code>shoe_width</code>

функции	= replicate('E', 5)	= 'EEEEEE'
Сцепление строк	full_name = 'Fred' + ' ' + 'Simpson'	full_name = 'Fred Simpson'
Функции работы с датами	week = datepart(w k, '5/22/99')	week = 21

Примечание. Выражение с функцией **getdate()** не может быть оптимизировано.

Эти преобразования позволяют оптимизатору использовать значения гистограммы для столбца вместо значений селективности по умолчанию.

Исключения:

- функция **getdate()**;
- большинство системных функций, таких как **object_id** и **object_name**.

Они не преобразуются в литеральные значения перед началом оптимизации.

Транзитивное замыкание аргументов поиска

Во время предварительной обработки к аргументам поиска применяется транзитивное замыкание. Например, следующий запрос соединяет таблицы **titles** и **titleauthor** в **title_id** и содержит аргумент поиска для **titles.title_id**:

```
select au_lname, title from titles t, titleauthor ta, authors a
where t.title_id = ta.title_id
      and a.au_id = ta.au_id
      and t.title_id = 'T81002'
```

Этот запрос оптимизируется так, как если бы он также содержал аргумент поиска для **titleauthor.title_id**:

```
select au_lname, title from titles t, titleauthor ta, authors a
where t.title_id = ta.title_id
      and a.au_id = ta.au_id
```

```
and t.title_id = 'T81002'  
and ta.title_id = 'T81002'
```

С этой дополнительной инструкцией оптимизатор может использовать статистику индекса для **titles.title_id**, чтобы оценить количество подходящих строк в таблице **titleauthor**. Более точные оценки стоимости улучшают выбор индекса и порядка соединения.

Транзитивное замыкание соединений

Во время предварительной обработки к столбцам, по которым производится соединение, применяется транзитивное замыкание (для обычных соединений по равенству), если транзитивное замыкание соединений включено на уровне сервера или сеанса. Следующий запрос задает соединение по равенству столбцов **t1.c11** и **t2.c21**, а также соединение по равенству столбцов **t2.c21** и **t3.c31**:

```
select * from t1, t2, t3  
where t1.c11= t2.c21  
and t2.c21 = t3.c31  
and t3.c31 =1
```

Без транзитивного замыкания соединений рассматриваются только следующие порядки соединения: **(t1, t2, t3)**, **(t2, t1, t3)**, **(t2, t3, t1)** и **(t3, t2, t1)**.

Добавив соединение **t1.c11 = t3.c31**, оптимизатор расширяет круг доступных порядков соединения вариантами **(t1, t3, t2)** и **(t3, t1, t2)**. Транзитивное замыкание аргументов поиска применяет условие **t3.c31 = 1** к столбцам **t1.c11** и **t2.c21**.

Транзитивное замыкание используется только для обычных соединений по равенству, как показано выше. Транзитивное замыкание соединений не выполняется для следующих элементов:

- соединений, основанных не на равенстве, например **t1.c1 > t2.c2**;
- соединений по равенству, которые содержат выражение, например **t1.c1 = t2.c1 + 5**;
- соединений по равенству, соединенных инструкцией **or**;
- внешних соединений, например **t1.c11 != t2.c2**, или **left join**, или **right join**;
- соединений в подзапросах;
- соединений, использующихся для проверки ссылочной целостности или инструкции **with check option** для представлений;
- столбцов с несовместимыми типами данных.

Трансформация предикатов

Трансформация предикатов увеличивает количество вариантов выбора, доступных оптимизатору. Она заключается в том, что в запрос добавляются инструкции, которые могут быть оптимизированы.

Это делается путем выборки инструкций из блоков предикатов, связанных ключевым словом **or**, в инструкции, связанные ключевым словом **and**. Эти дополнительные инструкции увеличивают количество вариантов выполнения запроса. Исходные предикаты **or** сохраняются для обеспечения правильности запроса.

Пример исходного запроса:

```
select p.pub_id, t.price
  from publishers p, titles t
 where (
      t.pub_id = p.pub_id
      and t.type = 'travel'
      and t.price between 15 and 30
      and p.pub_id in ('P220', 'P583', 'P780')
    )
    or
    (
      t.pub_id = p.pub_id
      and t.type = 'business'
      and t.price between 15 and 50
      and p.pub_id in ('P651', 'P066', 'P629')
    )
```

При трансформации предикатов происходит следующее:

1. Выбираются простые предикаты (соединения, аргументы поиска и списки **in**), которые в точности одинаковы в каждой инструкции **or**. В примере запроса эта инструкция в точности повторяется в каждом блоке и поэтому выбирается: **t.pub_id = p.pub_id**
2. Перед трансформацией предикатов инструкции **between** преобразуются в инструкции со сравнением на “больше или равно” и на “меньше или равно”. В примере запроса, приведенном выше, инструкция **between 15** используется в обоих блоках запроса (хотя границы диапазонов различаются). Эквивалентная инструкция выбирается на этапе 1: **t.price >=15**. Для одной и той же таблицы выбираются аргументы поиска; все элементы, которые ссылаются на одну и ту же таблицу, во время расширения рассматриваются как один предикат. Столбцы **type** и **price** находятся в таблице **titles t**, поэтому выбираются

следующие инструкции:

```
(t.type = 'travel' and t.price >=15 and t.price <= 30)
or
(t.type = 'business' and t.price >= 15 and t.price <= 50)
```

3. Выбираются списки **in** и инструкции **or**. Если внутри одного из блоков есть несколько списков **in** для таблицы, извлекается только первый. Для приведенного примера запроса выбираются следующие списки:

```
p.pub_id in ('P220', 'P583', 'P780')
or
p.pub_id in ('P651', 'P066', 'P629')
```

4. Эти этапы могут перекрываться и на них могут извлекаться одинаковые инструкции, поэтому все дубликаты удаляются.
5. Изучается каждый созданный элемент, чтобы определить, можно ли его использовать как оптимизированный аргумент поиска или инструкцию соединения. Сохраняются только те элементы, которые полезны для оптимизации запроса.
6. Дополнительные инструкции добавляются к существующим инструкциям запроса.

Результат обработки :

```
select p.pub_id, t.price
  from publishers p, titles t
 where t.pub_id = p.pub_id
    and t.price >= 15
    and (
      (
        t.type = 'travel'
        and t.price <=30
        and p.pub_id in ('P220', 'P583', 'P780')
      )
    or
      (
        t.type = 'business'
        and t.price <=50
        and p.pub_id in ('P651', 'P066', 'P629')
      )
    )
```

При трансформации предикатов инструкции, связанные ключевым словом **and**, извлекаются из блоков инструкций, связанных ключевым словом **or**, как в вышеприведенном примере.

Извлекаются только те инструкции, которые встречаются во всех блоках, заключенных в скобки.

Если бы в одном из блоков из вышеприведенного примера, связанных ключевым словом **or**, была инструкция, которая не присутствовала бы в другом блоке, эта инструкция не была бы извлечена.

Рекомендации по созданию аргументов поиска

При написании аргументов поиска для запросов придерживайтесь следующих рекомендаций:

- Избегайте указания функций, арифметических операций и других выражений на стороне столбцов инструкций поиска. По возможности переносите функции и другие операции на сторону выражений инструкции.
- Избегайте использования несовместимых типов данных для столбцов, по которым будет осуществляться соединение, а также для переменных и параметров, используемых в качестве аргументов поиска.
- В качестве аргумента поиска используйте начальный столбец составного индекса. Оптимизация вторичных ключей дает менее высокую производительность.
- Используйте все аргументы поиска, какие только возможно, чтобы предоставить оптимизатору как можно большее их количество.
- Если в запросе более **102** предикатов для таблицы, помещайте потенциально наиболее полезные инструкции в начало запроса, так как во время оптимизации для каждой таблицы используются только первые **102** (для отбора строк используются все условия поиска)
- Некоторые запросы, содержащие условие **>** (больше), могут выполняться лучше, если их переписать с условием **>=** (больше или равно). Например, следующий запрос с индексом по столбцу `int_col` применяет индекс для нахождения первого значения, когда `int_col` равняется 3, а затем продолжает сканирование, чтобы найти первое значение, которое превышает 3. Если строк, для которых `int_col` равно 3, много, серверу придется просканировать много страниц, чтобы найти первую строку, где `int_col` превышает 3:

```
select * from table1 where int_col > 3
```

Более эффективным будет запрос, записанный следующим образом:

```
select * from table1 where int_col >= 4
```

Эта оптимизация более сложна при использовании символьных строк и данных с плавающей точкой. Необходимо знать используемые данные.

- Проверьте выходные данные команды **set showplan**, чтобы узнать, какие ключи и индексы используются.
- Если индекс, применения которого вы ожидали, не используется, необходимо проверить, рассматривает ли оптимизатор этот индекс.

Аргументы поиска и полезные индексы

Важно различать предикаты инструкций **where** и **having**, которые могут использоваться для оптимизации запроса, и предикаты, которые используются позже, во время обработки запроса, для фильтрации возвращаемых строк.

Аргументы поиска можно использовать для определения пути доступа к строкам данных, когда столбец в инструкции **where** совпадает с начальным индексным ключом. Индекс можно применять для поиска и извлечения подходящих строк данных. Когда строка найдена в кэше данных или считана в кэш с диска, применяются все остальные инструкции.

Например, если в таблице **authors** есть индекс по столбцу **au_lname** и другой индекс по столбцу **city**, любой из них можно использовать для поиска подходящих строк для этого запроса:

```
select au_lname, city, state
  from authors
 where city = 'Washington'
    and au_lname = 'Catmull'
```

Оптимизатор использует статистику по страницам данных и индексов (в том числе гистограммы, количество строк в таблице, количество уровней индексов и коэффициенты кластеризации), чтобы определить, какой индекс обеспечит наиболее дешевый доступ. Индекс, обеспечивающий самый дешевый доступ к страницам данных, выбирается и используется для выполнения запроса, а другая инструкция применяется к строкам данных после того, как к ним был произведен доступ.

Синтаксис аргументов поиска

Аргументы поиска (search arguments, SARG) – это выражения в одной из следующих форм:

1. <столбец> <оператор> <выражение>
2. <выражение> <оператор> <столбец>
3. <столбец> is null

где:

- **столбец** – это только имя столбца. Если к имени столбца добавляются функции, выражения или конкатенация, индекс для этого столбца не может быть использован.
- **оператор** должен быть одним из следующих: =, >, <, >=, <=, !=, !<, !>, !=, is null
- **выражение** – это или константа, или выражение, которое возвращает значение-константу. Оптимизатор по-разному использует статистику индексов в зависимости от того, известно

ли значение выражения во время компиляции.

- Если выражение – это известная константа или оно может быть преобразовано в известную константу во время предварительной обработки, его можно сравнить со значениями гистограммы, хранимыми для индекса, для получения точных оценок строк.
- Если во время компиляции значение выражения неизвестно, оптимизатор оценивает количество строк, возвращаемых запросом, с помощью общей плотности. Значения, присвоенные переменным в пакете запросов, или параметры, установленные в хранимой процедуре, неизвестны до времени выполнения.
- Если тип данных выражения несовместим с типом данных столбца, индекс использовать нельзя, и он не рассматривается.

Операторы неравенства

Операторы неравенства (< > и !=) – это особый случай. Если столбец индексирован, оптимизатор ищет покрывающие некластерные индексы и применяет сплошное сканирование индекса, если индекс покрывает запрос. Однако если индекс не покрывает запрос, производится сканирование таблицы.

Примеры аргументов поиска

Ниже приведены несколько примеров инструкций, которые могут быть полностью оптимизированы. Если по этим столбцам есть статистика, она может использоваться для оценки количества строк, которые вернет запрос. Если по столбцам есть индексы, они могут применяться для доступа к данным:

```
au_lname = 'Bennett'  
price >= 12.00  
advance > 10000 and advance < 20000  
au_lname like 'Ben%' and price > 12.00
```

Следующие аргументы поиска не могут быть оптимизированы:

```
advance * 2 = 5000 /* выражение, содержащее столбец, недопустимо */  
substring(au_lname,1,3) = 'Ben' /* функция от столбца */
```

Эти две инструкции можно оптимизировать, если записать их в такой форме:

```
advance = 5000/2  
au_lname like 'Ben%'
```

Рассмотрим следующий запрос с единственным индексом по столбцу au_lname:

```
select au_lname, au_fname, phone
  from authors
 where au_lname = 'Gerland'
       and city = 'San Francisco'
```

Инструкция

```
au_lname = 'Gerland'
```

подходит для того, чтобы быть аргументом поиска при указанных условиях:

- Есть индекс по столбцу, указанному в инструкции (в данном случае **au_lname**).
- В запросе этот столбец не участвует в функциях или других операциях.
- Оператор является допустимым оператором SARG.
- Тип данных константы совпадает с типом данных столбца.

```
city = 'San Francisco'
```

Эта инструкция соответствует всем критериям, перечисленным выше, кроме первого: по столбцу **city** нет индекса. В этом случае в запросе используется индекс по столбцу **au_lname**. Все страницы данных с подходящей фамилией переносятся в кэш, и производится проверка на соответствие города в каждой подходящей строке критерию поиска.

Синтаксис и обработка соединений

Инструкции соединения записываются в следующей форме:

```
таблица1.имя_столбца <оператор> таблица2.имя_столбца
```

Операторами соединения являются:

```
=, >, >=, <, <=, !>, !<, !=, <>, *=, =*
```

Также возможны формы записи

```
таблица1 [ left | right ] join таблица2
on имя_столбца = имя_столбца
```

```
таблица1 inner join таблица2
on имя_столбца = имя_столбца
```

Когда соединения оптимизируются, оптимизатор может учитывать только индексы по именам столбцов. Любой тип оператора или выражения в сочетании с именем столбца означает, что оптимизатор не будет оценивать использование индекса для столбца в качестве метода доступа.

Если типы столбцов соединения несовместимы, оптимизатор может рассматривать индекс только по одному столбцу.

Как обрабатываются соединения

Если оптимизатор создает план для запроса с соединением, он выполняет следующие действия:

- Оценивает индексы для каждой таблицы, подсчитывая стоимость операций ввода-вывода, необходимых для каждого возможного индекса и для сканирования таблицы.
- Определяет порядок соединения, основывая свое решение на оценках общей стоимости для возможных порядков соединения. Оптимизатор оценивает стоимость выполнения соединений как методом вложенных циклов, так и методом “сортировка-слияние”.
- Если для внутренней таблицы соединения нет полезного индекса, оптимизатор может выбрать построение временного индекса – этот процесс называется переформатированием.
- Определяет размер блока ввода-вывода и стратегию кэширования.
- Сравнивает стоимость последовательного и параллельного выполнения, если включена параллельная обработка запроса.

Факторы, определяющие стоимость выборок из одной таблицы (в частности, надлежащее индексирование, селективность аргументов поиска и плотность ключей), приобретают дополнительную важность для соединений.

Аргументы поиска и соединения для таблицы

Если столбцы таблицы входят как в аргументы поиска, так и в соединения, то, чтобы более точно оценить количество строк, во время оценки стоимости соединений используются совместные значения селективности столбцов.

В следующем примере таблицы **authors** и **stores** соединяются по столбцам **city** и **state**. Столбец **authors.state** является аргументом поиска, поэтому транзитивное замыкание добавляет аналогичный аргумент поиска и для столбца **stores.state**:

```
select au_fname, au_lname, stor_name
from authors a, stores s
```

```
where a.city = s.city  
and a.state = s.state  
and a.state = 'GA'
```

Если у каждой таблицы есть индекс по столбцу **city**, но нет статистики по столбцу **state**, оптимизатор использует селективность аргумента поиска по умолчанию (10 %), объединенную с общей плотностью для столбца **city**. Такая оценка завышает количество строк, соответствующих аргументу поиска для этого запроса; однако если аргументу поиска по столбцу **state** соответствует большее количество строк, оценка количества строк будет занижена. Если статистика по столбцу **state** есть в каждой таблице, то оценка количества подходящих строк, а также общая оценка стоимости запроса с соединением становится более точной.

Несоответствие типов данных и оптимизация запроса

Частой проблемой в случаях, когда запросам не удастся использовать индексы, как ожидалось, является несоответствие типов данных. Несоответствия типов данных происходят:

- Когда используются инструкции поиска, содержащие переменные или параметры хранимых процедур, тип данных которых отличается от типа столбца, например:

```
where int_col = @money_parameter
```

- В запросах с соединением, когда столбцы, по которым производится соединение, имеют разные типы данных, например:

```
where tableA.int_col = tableB.money_col
```

Несоответствия типов данных приводят к проблемам с оптимизацией, так как они не позволяют оптимизатору выбирать индекс. Наиболее часто встречающиеся проблемы возникают по следующим причинам:

- из-за сравнений между целочисленными типами, **int**, **smallint** и **tinyint**;
- из-за сравнений между типами **money** и **smallmoney**;
- из-за сравнений между типами **datetime** и **smalldatetime**;
- из-за сравнений между типами **numeric** и **decimal** разной точности и масштаба;
- из-за сравнений между типами **numeric** или **decimal** и столбцами **integer** или **money**.

Чтобы избежать проблем, при создании таблиц используйте один тип данных (с одинаковой точностью и масштабом) для столбцов, которые являются вероятными кандидатами на соединение. Используйте соответствующий тип данных для любых переменных и параметров хранимых процедур, применяемых в качестве аргументов поиска. В следующих разделах подробно рассматриваются правила и рассуждения, которые применяются в случаях использования разных типов данных, а также даются некоторые советы по устранению проблем.

Основные единицы оценки стоимости

Когда оптимизатор оценивает стоимость запроса, два фактора, которые он принимает в расчет, это стоимость физического ввода-вывода, то есть чтения страниц с диска, и стоимость логического ввода-вывода, то есть поиска страниц в кэше данных.

Оптимизатор устанавливает значение **18** для стоимости физического ввода-вывода и **2** – для стоимости логического ввода-вывода. Это относительные единицы стоимости, и они не представляют такие единицы времени, как миллисекунды или такты системных часов. Эти единицы используются в формулах подсчета стоимости, причем сначала идет стоимость физического ввода-вывода, а затем – логического ввода-вывода. Общая стоимость доступа к таблице может быть выражена формулой:

$$\begin{aligned} \text{Стоимость} &= \text{Все операции физического ввода-вывода} * 18 \\ &+ \text{Все операции логического ввода-вывода} * 2 \end{aligned}$$

Специальные методики оптимизации

Средства, рассматриваемые здесь, позволяют переопределять решения, принятые оптимизатором ASE, и могут оказать крайне отрицательное влияние на производительность в случае неверного использования. Необходимо понимать их влияние на производительность как индивидуального запроса, так и всей системы.

Оптимизатор по стоимости Adaptive Server в большинстве ситуаций формирует хорошие планы запросов. Но иногда оптимизатор не выбирает подходящий индекс для оптимальной производительности или выбирает не самый оптимальный порядок соединения, и необходимо самому указать метод доступа для запроса. Возможности, описываемые в этой главе, позволяют сделать это.

Кроме того, во время настройки может возникнуть желание увидеть результаты другого порядка соединения, размера блока ввода-вывода или стратегии кэширования. Некоторые из этих возможностей позволяют задавать обработку запроса или стратегию доступа без дорогостоящих изменений конфигурации.

Adaptive Server предусматривает средства и инструкции запроса, влияющие на оптимизацию запроса, и дополнительные средства анализа запроса, позволяющие понять, почему оптимизатор делает тот или иной выбор.

Указание вариантов выбора для оптимизатора

Adaptive Server позволяет указывать следующие варианты оптимизации путем добавления

команд в пакет запросов или в текст запроса:

- количество таблиц, обрабатываемых за раз во время оптимизации соединения;
- порядок таблиц в соединении;
- индекс, используемый для доступа к таблице;
- размер блока ввода-вывода;
- стратегия кэширования;
- степень параллелизма.

В отдельных случаях оптимизатору не удастся выбрать лучший план. В некоторых из этих случаев стоимость плана, который он выбирает, лишь немного больше стоимости “лучшего” плана, поэтому следует тщательно сравнить затраты на поддержание принудительно заданных параметров с более медленным выполнением по менее оптимальному плану.

Команды, указывающие порядок соединения, индекс, размер блока ввода-вывода или стратегию кэширования, в сочетании с командами отчета о запросе, такими как **set statistics io** и **set showplan**, помогают определить, почему оптимизатор делает тот или иной выбор.

Эти возможности в основном предназначены для настройки и экспериментирования, а не для долговременных решений проблем оптимизации.

Рассмотрим первые три варианта оптимизации.

Указание порядка таблиц в соединениях

Adaptive Server оптимизирует порядки соединения для минимизации операций ввода-вывода. В большинстве случаев порядок, выбираемый оптимизатором, не соответствует порядку, указанному в инструкциях **from** команды **select**. Чтобы заставить Adaptive Server обращаться к таблицам в порядке их перечисления в запросе, введите:

```
set forceplan [on|off]
```

В этом случае оптимизатор по-прежнему будет выбирать наилучший метод доступа для каждой таблицы. Однако он может использовать не те индексы, которые он использовал бы при другом порядке соединения, или не сможет использовать существующие индексы.

Эту команду можно использовать как средство отладки, если другие средства анализа запроса свидетельствуют о том, что оптимизатор выбирает не лучший порядок соединения. Всегда следует проверять, что порядок, выбираемый принудительно, сокращает количество операций ввода-вывода и логических чтений. Для этого нужно выполнить команду **set statistics io on** и

сравнить ввод-вывод с параметром **forceplan** и без него.

Если вы используете параметр **forceplan**, то необходимо регулярно проверять, что этот параметр по-прежнему нужен использующим его запросам и процедурам для повышения производительности.

Параметр **forceplan** можно включать в текст хранимых процедур. Команда **set forceplan** принудительно указывает только порядок соединения, но не его тип. Команды, указывающей тип соединения, не существует. Выполнение соединений методом слияния можно запретить на уровне сервера или сеанса.

Для принудительного порядка соединения характерны следующие риски:

- Неверное использование может резко повысить стоимость запросов. Всегда тщательно проверяйте запрос командой **set statistics io** с командой **set forceplan** и без нее.
- Параметр **forceplan** требует сопровождения. Необходимо регулярно проверять запросы и хранимые процедуры, содержащие параметр **forceplan**. Кроме того, будущие версии Adaptive Server могут решить проблемы, заставляющие вас принудительно указывать индексы, поэтому после установки каждой новой версии проверяйте все запросы, использующие принудительные планы выполнения.

Прежде чем использовать параметр **forceplan**, выполните следующее:

- Проверьте отчет команды **showplan**, чтобы определить, используются ли индексные ключи так, как ожидалось.
- Выполните команду **update statistics** для обновления статистики по индексу.
- Выполните команду **update statistics** для формирования статистики по аргументам поиска в инструкциях поиска запроса, не содержащих индексированных столбцов, особенно для младших ключей в составных индексах.
- Если запрос соединяет более четырех таблиц, выполните команду **set table count**, чтобы увидеть, не улучшит ли она порядок соединения.

Указание количества таблиц, рассматриваемых оптимизатором

Adaptive Server оптимизирует соединения, рассматривая перестановки от двух до четырех таблиц за раз. Если кажется, что для запроса с соединением выбирается неэффективный порядок соединения, можно увеличить количество таблиц, рассматриваемых одновременно, с помощью команды **set table count**.

set table count - Задаёт количество таблиц, которые сервер Adaptive Server обрабатывает за один раз при оптимизации соединения. Значение, используемое по умолчанию, зависит от количества таблиц в соединении следующим образом:

Количество таблиц, участвующих в соединении	Количество таблиц, обрабатываемых за один раз
2-25	4
26-37	3
38-50	2

Допустимые значения: **0 – 8**. Значение **0** задаёт поведение по умолчанию. Значения выше **8** по умолчанию заменяются на **8**. Установка параметра **table count** может улучшить оптимизацию некоторых запросов с соединениями, но увеличивает время компиляции.

Например, чтобы при оптимизации одновременно рассматривались четыре таблицы, введите:

```
set table count 4
```

Команда **dbcc traceon(310)** сообщает количество рассматриваемых таблиц за раз. Чем меньше это значение, тем меньше вероятность того, что оптимизатор рассмотрит все возможные порядки соединения. Увеличение количества таблиц, рассматриваемых за раз при выборе порядка соединения, может значительно увеличить время оптимизации запроса.

Поскольку время, необходимое для оптимизации запроса, увеличивается с каждой дополнительной таблицей, параметр **set table count** наиболее полезен в случаях, когда экономия затрат на выполнение благодаря улучшенному порядку соединения перевешивает дополнительное время, затрачиваемое на оптимизацию. К таким случаям относятся следующие:

- если оптимизируются хранимые процедуры, которые будут выполняться много раз после того, как план окажется в процедурном кэше;
- если сформированные абстрактные планы будут сохранены для последующего использования.

Используйте параметр **statistics time** для проверки времени разбора и компиляции и параметр **statistics io**, чтобы убедиться в том, что улучшенный порядок соединения сокращает количество физических и логических операций ввода-вывода.

Если увеличение значения параметра **table count** улучшает оптимизацию соединения, но недопустимо увеличивает время использования процессора, перепишите инструкцию **from** в запросе, перечислив таблицы в порядке соединения, указанном в отчете **showplan**, и примените

параметр **forceplan** для выполнения запроса. При регламентных проверках производительности нужно также проверять, что принудительно используемый порядок соединения по-прежнему повышает производительность.

Указание индекса для запроса

В операторах **select**, **update** и **delete** с помощью инструкции (`index имя_индекса`) можно указать индекс, который будет использоваться в запросе. Также можно заставить запрос выполнить сканирование таблицы, указав имя таблицы. Вот синтаксис таких операторов:

```
select список_выборки
  from имя_таблицы [псевдоним] (index {имя_индекса | имя_таблицы } )
    [, имя_таблицы ...]
where ...

delete имя_таблицы
  from имя_таблицы [псевдоним] (index {имя_индекса | имя_таблицы } ) ...

update имя_таблицы set имя_столбца = value
  from имя_таблицы [псевдоним] (index {имя_индекса | имя_таблицы } ) ...
```

Например:

```
select pub_name, title
  from publishers p, titles t (index date_type)
where p.pub_id = t.pub_id
     and type = 'business'
     and pubdate > '1/1/93'
```

Указание индекса в запросе может помочь, если есть подозрение, что оптимизатор выбирает не самый оптимальный план запроса. Если указана эта инструкция, необходимо выполнить следующее:

- Проверить отчет **statistics io** для запроса, чтобы узнать, требует ли принудительно выбираемый индекс меньше операций ввода-вывода, чем индекс, выбираемый оптимизатором.
- Протестировать полный диапазон допустимых значений для инструкций запроса, особенно в следующих случаях:
 - при настройке запросов к таблицам, в которых данные распределены несимметрично;
 - при выполнении запросов по диапазону, поскольку выбор методов доступа для этих запросов существенно зависит от размера диапазона.

Использовать эту возможность следует только после проверки, чтобы быть уверенным в том, что запрос выполняется лучше с указанием индекса. В случае включения индекса в запрос следует регулярно проверять, по-прежнему ли принудительно заданный план лучше, чем другие варианты, выбранные оптимизатором.

При указании индексов возникают следующие риски:

- Из-за изменений в распределении данных принудительно используемый индекс может стать менее эффективным, чем другие варианты.
- Если удалить принудительно используемый индекс, все запросы и процедуры, в которых он указан, будут сообщать, что этот индекс не существует, и будут оптимизироваться с использованием лучшего альтернативного метода доступа.
- Увеличивается сопровождение, поскольку все запросы, использующие индекс, необходимо периодически проверять. Кроме того, будущие версии Adaptive Server могут решить проблемы, заставляющие принудительно указывать индексы, поэтому после установки каждой новой версии проверяйте все запросы, использующие такие индексы.

Прежде чем указывать индекс в запросах, выполните следующее:

- Проверьте отчет команды **set showplan** на предмет сообщения “Keys are” (ключи), чтобы убедиться, что индексные ключи используются, как планировалось.
- Выполните команду **dbcc traceon(302)** для поиска других проблем оптимизации.
- Выполните команду **update statistics** по этому индексу.
- Если индекс – составной, выполните команду **update statistics** по младшим ключам в индексе, если они используются в качестве аргументов поиска. Это может заметно улучшить оценки стоимости, выполняемые оптимизатором. Создание статистических данных для других столбцов, часто используемых для инструкций поиска, также может улучшить оценки.

Использование команды **set statistics**

Команда **set statistics io** выдает информацию о физических и логических операциях ввода-вывода и о том, сколько раз происходило обращение к таблице. Выходные данные команды **set statistics io** следуют за результатами запроса и дают реальную картину ввода-вывода, осуществленного этим запросом.

Если параметр **statistics io** установлен в режим **ON**, для каждой таблицы в запросе (включая рабочие) выводится одна строка с несколькими значениями для страниц, считанных запросом, плюс одна строка с общим количеством операций записи. Если системный администратор установил ограничения на ресурсы, выходные данные **statistics io** содержат дополнительную

строку об общей стоимости ввода-вывода для данного запроса. Следующий пример показывает выходные данные **statistics io** для запроса с включенными ограничениями на ресурсы:

```
select avg(total_sales) from titles
```

```
Table: titles scan count 1, logical reads: (regular=656 apf=0 total=656),  
physical reads: (regular=444 apf=212 total=656), apf IOs used=212  
Total actual I/O cost for this command: 13120.  
Total writes for this command: 0
```

Есть четыре главных компонента статистики ввода-вывода, выводимой при установке параметра **statistics io**:

1. общее число операций записи;
2. статистика чтения;
3. фактическая стоимость ввода-вывода;
4. имя таблицы и “количество сканирований”.

Общая фактическая стоимость ввода-вывода

Если установлены ограничения на ресурсы, выходные данные команды `set statistics io` содержат строку “Total actual I/O cost”. Общая фактическая стоимость ввода-вывода Adaptive Server – безразмерная величина. Стоимость запроса определяется по формуле, которую мы уже рассматривали выше:

$$\begin{aligned} \text{Стоимость} &= \text{Все физические операции ввода-вывода} * 18 \\ &+ \text{Все логические операции ввода-вывода} * 2 \end{aligned}$$

Эта формула умножает “стоимость” логического ввода-вывода на количество операций логического ввода-вывода, а “стоимость” физического ввода-вывода – на количество операций физического ввода-вывода.

Для предыдущего примера, выполняющего 656 операций физического чтения и 656 операций логического чтения, общая стоимость, выводимая в строке Total actual I/O cost, равна 13120 ($656 * 2 + 656 * 18$).

Статистика операций записи

В режиме **statistics io** сообщается общее количество буферов, записанных командой. Для запросов, выполняющих только чтение, запись происходит только в том случае, если грязные страницы пересекают метку стирки в кэше и тем самым записываются на диск. Для запросов,

изменяющих данные, может выполняться только одна операция записи (запись журнальной страницы), так как измененные страницы остаются на стороне кэша данных.

Статистика операций чтения

Установка параметра `statistics io` позволяет оценить количество операций логического и физического чтения для всех таблиц и индексов, включенных в запрос, в том числе рабочих таблиц. Информация о вводе-выводе для индексов входит в информацию о вводе-выводе для таблицы.

В таблице приведены значения, сообщаемые для операций логического и физического чтения, если параметр **`statistics io`** установлен в **ON**:

Выходные данные	Описание
<i>Операции логического чтения</i>	
regular	Сколько раз нужная страница была найдена в кэше. В это число не входят страницы, загруженные операцией асинхронной предварительной выборки (asynchronous prefetch, APF).
apf	Сколько раз были найдены в кэше страницы, помещенные в него асинхронной предварительной выборкой.
total	Общее количество операций логического чтения(сумма <code>regular</code> и <code>apf</code>).
<i>Операции физического чтения</i>	
regular	Сколько раз буфер загружался в кэш операциями обычного асинхронного ввода-вывода.
apf	Общее количество операций физического чтения (сумма <code>regular</code> и <code>apf</code>).
total	Общее количество операций физического чтения (сумма <code>regular</code> и <code>apf</code>).
apf IOs used	Количество буферов, которые были загружены в кэш операциями

	APF и страницы которых использовались во время запроса.
--	---

Статистика, выводимая при установке параметра `statistics io` при выполнении запроса без использования индекса, а затем того же запроса, но с использованием индекса, показывает, насколько хорошие индексы необходимы для повышения производительности запросов и системы. Покажем это на примере следующего запроса:

```
select title from titles where title_id = 'T5652'
```

Результаты для запроса без использования индекса

Если по столбцу **title_id** нет индекса, для запроса возвращаются следующие значения (размер блока ввода-вывода равен 2 КБ):

```
Table: titles scan count 1, logical reads:(regular=624
apf=0 total=624), physical reads:(regular=230 apf=394
total=624), apf IOs used=394
Total actual I/O cost for this command: 12480.
Total writes for this command: 0
```

Это означает следующее:

- Запрос осуществил всего 624 логических операции ввода-вывода. Все они были обычными (regular).
- Запрос осуществил 624 операции физического чтения. Из них 230 были обычными операциями асинхронного чтения и 394 – операциями предварительной выборки.
- Все страницы, считанные операциями предварительной выборки, были использованы в запросе.

Результаты для запроса с использованием индекса

Если по столбцу **title_id** построен кластерный индекс, то для того же запроса будут выведены следующие значения (размер блока ввода-вывода по-прежнему равен 2КБ):

```
Table: titles scan count 1, logical reads: (regular=3 apf=0 total=3),
physical reads: (regular=3 apf=0 total=3), apf IOs used=0
Total actual I/O cost for this command: 60.
Total writes for this command: 0
```

Это означает следующее:

- Запрос выполнил 3 операции логического чтения.
- Запрос выполнил 3 операции физического чтения: 2 операции чтения для страниц индекса 1 – для страницы данных.

Количество сканирований

При включении параметра **statistics io** выводится число обращений запроса к определенной таблице. Под “сканированием” (scan) понимается любой из следующих методов доступа:

- Сканирование таблицы.
- Доступ через кластерный индекс. Отдельным сканированием считается каждый случай, когда запрос начинает сканирование с корневой страницы индекса и переходит по указателям на страницы данных.
- Доступ через некластерный индекс. Отдельным сканированием считается каждый случай, когда запрос начинает сканирование с корневой страницы индекса и переходит по указателям на листовые страницы индекса (если все необходимые данные находятся в индексе) или на страницы данных.
- Если запросы выполняются параллельно, каждое обращение рабочего процесса к таблице считается отдельным сканированием.

Запросы, для которых число сканирований равно 1

Примеры запросов, возвращающих число сканирований, равное 1:

- Точечный запрос:

```
select title_id
  from titles
 where title_id = 'T55522'
```
- Диапазонный запрос:

```
select au_lname, au_fname
  from authors
 where au_lname > 'Smith'
       and au_lname < 'Smythe'
```

Если столбцы, указанные в инструкциях **where** этих запросов, индексированы, то для сканирования таблиц в запросах могут использоваться индексы. В противном случае выполняется обычное сканирование таблиц. В обоих случаях для получения нужных строк достаточно одного сканирования таблицы.

Запросы с числом сканирований более 1

Примеры запросов, возвращающих число сканирований, большее 1:

- Параллельные запросы. В них каждый рабочий процесс выполняет одно сканирование.
- Запросы, инструкции where которых содержат индексированные столбцы и соединены

логическим оператором **or**. В них каждому условию, соединенному **or**, также соответствует одно сканирование. Если запрос использует “специальную стратегию OR” (**special OR strategy**), в отчет включается одно сканирование для каждого значения, указанного в условии поиска. Если запрос использует стратегию OR, количество сканирований равно количеству просматриваемых индексов плюс один (дополнительное сканирование для доступа к списку идентификаторов строк). Этот запрос использует специальную стратегию OR, поэтому для него количество просмотров будет равно 2, если таблица **titles** индексирована по столбцам **title_id** и **pub_id**:

```
select title_id
  from titles
 where title_id = 'T55522'
        or pub_id = 'P988'
```

Table: titles scan count 2, logical reads: (regular=149 apf=0 total=149), physical reads: (regular=63 apf=80 total=143), apf IOs used=80

Table: Worktable1 scan count 1, logical reads: (regular=172 apf=0 total=172), physical reads: (regular=0 apf=0 total=0), apf IOs

Выдается также информация по вводу-выводу для рабочей таблицы.

- Соединения, выполняемые методом вложенных циклов. В них число сканирований внутренних таблиц равно числу строк внешней таблицы, удовлетворяющих условию соединения. В следующем примере внешняя таблица **publishers** содержит три строки со значением столбца **state**, равным “NY”, поэтому внутренняя таблица **titles** будет сканироваться 3 раза:

```
select title_id
  from titles t, publishers p
 where t.pub_id = p.pub_id
        and p.state = 'NY'
```

Table: titles scan count 3, logical reads: (regular=442 apf=0 total=442), physical reads: (regular=53 apf=289 total=342), apf IOs used=289

Table: publishers scan count 1, logical reads: (regular=2 apf=0 total=2), physical reads: (regular=2 apf=0 total=2), apf IOs used=0

Этот запрос просматривает таблицу **publishers**, которая занимает только 2 страницы данных, поэтому выполняются две физические операции ввода-вывода. В таблице **publishers** условиям запроса удовлетворяют 3 строки, поэтому запрос сканирует таблицу **titles** 3 раза, используя индекс по столбцу **pub_id**.

- Соединения, выполняемые методом слияния, в которых внешняя таблица содержит повторяющиеся значения. В них сканирование выполняется для каждого значения из группы повторяющихся значений.

Запросы с числом сканирований, равным 0

Для многоэтапных и некоторых других типов запросов количество сканирований может быть равно 0. Вот несколько примеров таких запросов:

- запросы, выполняющие отложенные обновления;
- запросы **select...into**;
- запросы, создающие рабочие таблицы.

Использование команды **set showplan**

Утилита **showplan** отображает этапы, выполняемые для каждого запроса в пакете, ключи и индексы, которые используются для запроса, порядок соединений и специальные стратегии оптимизатора.

Для просмотра плана запроса нужно ввести следующую команду:

```
set showplan on
```

а для отключения вывода плана – следующую команду:

```
set showplan off
```

Команду **set showplan** можно использовать вместе с другими командами **set**.

Если нужно просмотреть план хранимой процедуры, не выполняя ее, выполните команду

```
set fmtonly.
```

Основные сообщения **showplan**

Описание сообщения о плане запроса, которые выводятся утилитой **showplan** для большей части операций **select**, **insert**, **update** и **delete**.

Заголовок плана запроса

```
QUERY PLAN FOR STATEMENT N (at line N)
```

Adaptive Server выводит одну такую строку для каждого запроса в пакете. Основная функция этого сообщения – визуальное отделение разделов выходных данных утилиты **showplan**. Номера строк

указываются для того, чтобы помочь пользователю сопоставить выходные данные с введенным им запросом.

Сообщение STEP

STEP N

Утилита `showplan` выводит сообщения “STEP N” для каждого запроса, где N – целое число. Первым сообщением для запроса является “STEP 1”. Для некоторых запросов Adaptive Server не может извлечь результаты за один этап и разбивает план запроса на несколько этапов. Например, если запрос включает инструкцию `group by`, Adaptive Server разбивает его как минимум на два этапа:

- Первый этап – выбор из таблицы строк, соответствующих условию запроса и помещение их в рабочую таблицу.
- Второй этап – выбор строк из рабочей таблицы.

В примере показан одношаговый запрос:

```
select au_lname, au_fname
  from authors
 where city = 'Oakland'
```

```
QUERY PLAN FOR STATEMENT 1 (at line 1).
```

STEP 1

```
The type of query is SELECT.
```

```
FROM TABLE
```

```
authors
```

```
Nested iteration.
```

```
Table Scan.
```

```
Forward scan.
```

```
Positioning at start of table.
```

```
Using I/O Size 2 Kbytes for data pages.
```

```
With LRU Buffer Replacement Strategy for data pages.
```

Сообщение о типе запроса

```
The type of query is тип_запроса.
```

Это сообщение описывает тип запроса для каждого этапа. Для большинства запросов, для которых требуется настройка, типом запроса является **SELECT**, **INSERT**, **UPDATE** или **DELETE**. Но типом запроса может быть любая команда Transact SQL, перед выполнением которой параметр **showplan** был установлен в **on**.

Ниже приведено сообщение для команды **create index**:

```
STEP 1
The type of query is CREATE INDEX.
TO TABLE
titleauthor
```

Сообщение FROM TABLE

```
FROM TABLE
{имя_таблицы | псевдоним}
```

Это сообщение указывает, из какой таблицы запрос выполняет чтение. На следующей после слов “FROM TABLE” строке выводится имя таблицы. Если в инструкции **from** для таблиц были указаны псевдонимы, они выводятся после имен таблиц. Если запросы создают и используют рабочие таблицы, в сообщении “FROM TABLE” выводятся их имена. Если запрос соединяет одну или несколько таблиц, порядок сообщений “FROM TABLE” соответствует тому порядку, в котором эти таблицы соединяются (план соединения выбирается оптимизатором).

Ниже приведен запрос, выполняющий соединение трех таблиц, и сообщения утилиты showplan для него, указывающие выбранный порядок соединения:

```
select a.au_id, au_fname, au_lname
  from titles t, titleauthor ta, authors a
 where a.au_id = ta.au_id
    and ta.title_id = t.title_id
    and au_lname = 'Bloom'
```

```
QUERY PLAN FOR STATEMENT 1 (at line 1).
```

```
STEP 1
```

```
The type of query is SELECT.
```

```
FROM TABLE
```

```
authors
```

```
a
```

```
Nested iteration.
```

```
Index : au_lname_ix
```

```
Forward scan.
```

```
Positioning by key.
```

```
Keys are:
```

```
au_lname ASC
```

```
Using I/O Size 2 Kbytes for index leaf pages.
```

```
With LRU Buffer Replacement Strategy for index leaf pages.
```

Using I/O Size 2 Kbytes for data pages.
 With LRU Buffer Replacement Strategy for data pages.

FROM TABLE
titleauthor
ta

Nested iteration.
 Index : at_ix
 Forward scan.
 Positioning by key.
 Index contains all needed columns. Base table will not be read.
 Keys are:
 au_id ASC
 Using I/O Size 2 Kbytes for index leaf pages.
 With LRU Buffer Replacement Strategy for index leaf pages.

FROM TABLE
titles
t

Nested iteration.
 Using Clustered Index.
 Index : title_id_ix
 Forward scan.
 Positioning by key.
 Index contains all needed columns. Base table will not be read.
 Keys are:
 title_id ASC
 Using I/O Size 2 Kbytes for index leaf pages.
 With LRU Buffer Replacement Strategy for index leaf pages.

Последовательность таблиц в этих выходных данных показывает, что оптимизатор выбрал следующий порядок соединения (этот порядок отличается от порядка, в котором таблицы перечислены в инструкциях **from** или **where**):

- Сначала определяются строки таблицы **authors**, удовлетворяющие инструкции поиска по столбцу **au_lname**.
- Затем эти строки соединяются с таблицей **titleauthor** (на основе условия соединения столбцов **au_id**).
- Наконец, таблица **titles** соединяется с таблицей **titleauthor** (на основе условия соединения столбцов **title_id**).

Сообщение Table scan

Table Scan

Это сообщение указывает, что запрос выполняет сканирование таблицы. В следующем примере показан запрос с типичным сканированием таблицы:

```
select au_lname, au_fname
  from authors
```

```
QUERY PLAN FOR STATEMENT 1 (at line 1).
```

```
STEP 1
```

```
The type of query is SELECT.
```

```
FROM TABLE
```

```
authors
```

```
Nested iteration.
```

```
Table Scan.
```

```
Forward scan.
```

```
Positioning at start of table.
```

```
Using I/O Size 16 Kbytes for data pages.
```

```
With LRU Buffer Replacement Strategy for data pages.
```

Сообщение Clustered index

```
Using Clustered Index
```

Это сообщение утилиты showplan указывает, что оптимизатор запросов для выборки строк использует кластерный индекс по таблице. В следующем примере показано использование кластерного индекса для выборки строк из таблицы:

```
select title_id, title from titles where title_id like 'T9%'
```

```
QUERY PLAN FOR STATEMENT 1 (at line 1).
```

```
STEP 1
```

```
The type of query is SELECT.
```

```
FROM TABLE
```

```
titles
```

```
Nested iteration.
```

```
Using Clustered Index.
```

```
Index : title_id_ix
```

```
Forward scan.
```

```
Positioning by key.
```

```
Keys are:
```

```
title_id ASC
```

```
Using I/O Size 16 Kbytes for index leaf pages.
```

```
With LRU Buffer Replacement Strategy for index leaf pages.
```

Using I/O Size 16 Kbytes for data pages.

With LRU Buffer Replacement Strategy for data pages.

Сообщения об имени индекса

Index : имя_индекса

Это сообщение указывает, что запрос использует индекс для выборки строк. Сообщение включает имя индекса. Если непосредственно перед сообщением стоит строка “Using Clustered Index”, то индекс – кластерный, в противном случае – некластерный. Ключи, которые используются для указания начальной позиции поиска, приводятся в сообщении “Keys are...”.

В следующем запросе показано использование некластерного индекса для поиска и выборки строк:

```
select au_id, au_fname, au_lname
  from authors
 where au_fname = 'Susan'
```

QUERY PLAN FOR STATEMENT 1 (at line 1).

STEP 1

The type of query is SELECT.

FROM TABLE

authors

Nested iteration.

Index : au_names_ix

Forward scan.

Positioning by key.

Keys are:

au_fname ASC

Using I/O Size 16 Kbytes for index leaf pages.

With LRU Buffer Replacement Strategy for index leaf pages.

Using I/O Size 2 Kbytes for data pages.

With LRU Buffer Replacement Strategy for data pages.

Сообщения о направлении сканирования

Forward scan.

Backward scan.

Эти сообщения указывают направление сканирования таблицы или индекса. Направление сканирования зависит от порядка сортировки, указанного при создании индекса, и порядка сортировки, указанного в инструкции **order by**.

Обратное сканирование используется, если инструкция **order by** содержит квалификатор **asc** или **desc** для индексных ключей, которые противоположны квалификатору, заданному в команде **create index**. Чтобы разрешить обратное сканирование, параметру конфигурации **allow backward scans** должно быть присвоено значение **1**. За сообщением о направлении сканирования выводятся сообщения о начальной позиции. В этих сообщениях сначала указывается имя ключа, используемого в запросе, а затем – ключевое слово “ASC” или “DESC”. Сообщения о прямом или обратном сканировании и сообщения о начальной позиции указывают начальную позицию сканирования. Начальная позиция может быть одной из следующих двух:

- На первом индексном ключе, соответствующем условию поиска, в начале таблицы или на первой странице цепочки листовых страниц. В этом случае поиск выполняется к концу индекса или таблицы.
- На последнем индексном ключе, соответствующем условию поиска, в конце таблицы или на последней странице цепочки листовых страниц. В этом случае поиск выполняется к началу индекса или таблицы.

Если значение параметра **allow backward scans** равно **0**, всегда будет использоваться прямое сканирование.

В следующем примере используется обратное сканирование:

```
select *
  from sysmessages
 where description like '%Optimized using%'
order by error desc
```

```
QUERY PLAN FOR STATEMENT 1 (at line 1).
```

```
STEP 1
```

```
The type of query is SELECT.
```

```
FROM TABLE
```

```
sysmessages
```

```
Nested iteration.
```

```
Table Scan.
```

```
Backward scan.
```

```
Positioning at end of table.
```

```
Using I/O Size 2 Kbytes for data pages.
```

```
With LRU Buffer Replacement Strategy for data pages.
```

В этом запросе, содержащем агрегатную функцию **max**, также используется обратное сканирование:

```
select max(error) from sysmessages
```

```
QUERY PLAN FOR STATEMENT 1 (at line 1).
```

STEP 1

The type of query is SELECT.

Evaluate Ungrouped MAXIMUM AGGREGATE.

FROM TABLE

sysmessages

Nested iteration.

Index : ncsysmessages

Backward scan.

Positioning by key.

Scanning only up to the first qualifying row.

Index contains all needed columns. Base table will not be read.

Keys are:

error ASC

Using I/O Size 2 Kbytes for index leaf pages.

With LRU Buffer Replacement Strategy for index leaf pages.

STEP 2

The type of query is SELECT.

Имеются другие сообщения возвращаемые командой **showplan**, которые можно изучить в руководстве.